Machine Learning

Deep learning: Single hidden layer neural networks

Sebastian Engelke

Master in Business Analytics

UNIVERSITÉ DE GENÈVE

Neural networks

- In recent years, the term `(artificial) neural networks` has caused a big hype in machine learning, especially in connection with `deep learning`.
- In the media, this method appears to be magical or `mysterious` and is often believed as being the solution to all prediction problems.

Neural networks

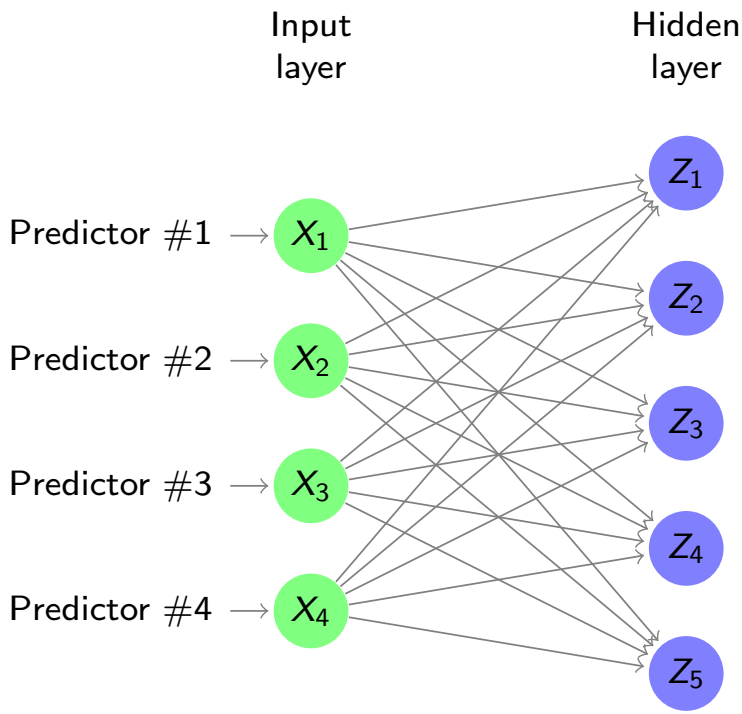
- In recent years, the term `(artificial) neural networks` has caused a big hype in machine learning, especially in connection with `deep learning`.
- In the media, this method appears to be magical or `mysterious` and is often believed as being the solution to all prediction problems.
- We will de-mystify this term and understand a `(deep) neural network` as what it is: a `highly flexible`, `non-linear` prediction method.
- Neural networks can be used for both `regression` and `classification`.

Single hidden layer neural network for regression

- In its simplest form, a neural network for predicting a quantitative Y is constructed as follows.
- The p inputs $X = (X_1, \ldots, X_p)$ are non-linearly combined to form m hidden variables by

$$Z_j = g(b_j^{(1)} + {w_j^{(1)}}^\top X), \quad j = 1, \ldots, m$$

where $w_j^{(1)} = (w_{j1}^{(1)}, \ldots, w_{jp}^{(1)}) \in \mathbb{R}^p$ and $b_j^{(1)} \in \mathbb{R}$ are called the weights and the bias, respectively, and the activation function g can introduce the non-lineartiy.

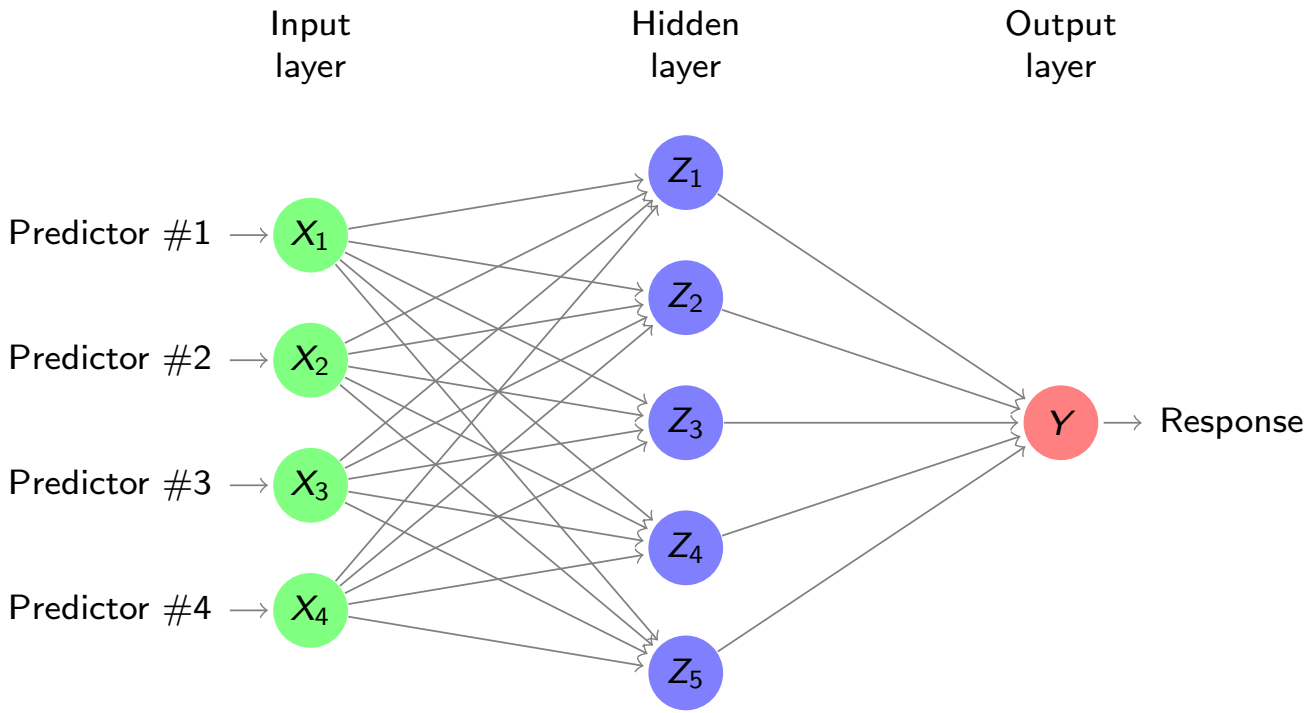


Single hidden layer neural network for regression

- The $Z = (Z_1, \ldots, Z_m)$ are then further combined to `model the output` Y as

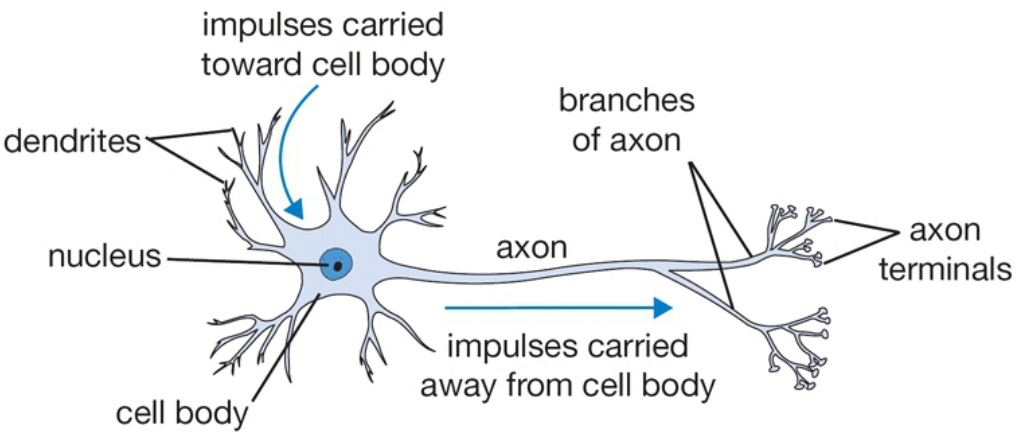
$$Z_j = g(b_j^{(1)} + {w_j^{(1)}}^\top X), \quad j = 1, \ldots, m,$$
$$Y \approx \widehat{f}(X) = h(b^{(2)} + {w^{(2)}}^\top Z).$$

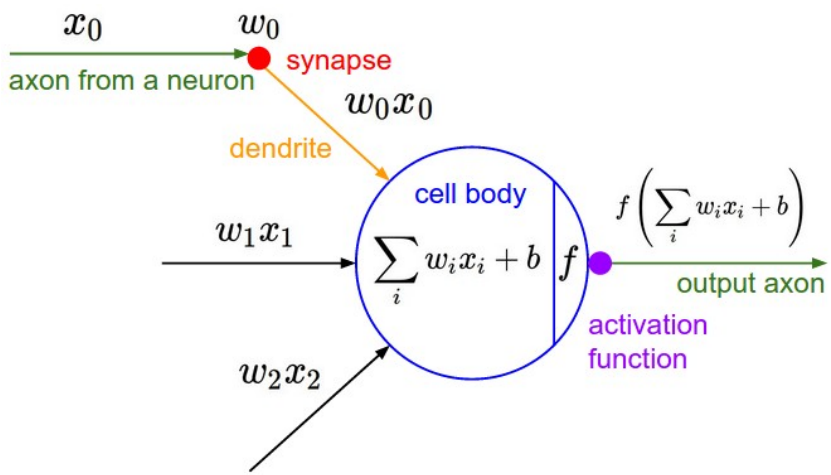
with weights $w^{(2)} = (w_1^{(2)}, \ldots, w_m^{(2)}) \in \mathbb{R}^m$ and bias $b^{(2)} \in \mathbb{R}$, and some function h.



Biological motivation and notation

- `Artificial neural network` are loosely motivated by the functioning of `neurons` in our brain.
- The `nomenclature` in this area of machine learning is therefore linked to `biological terms`.
- The nodes in the network are called `neurons`, they can be `activated` and they `fire` if the activation is large enough.





Single hidden layer neural network for regression

- In matrix notation, we can write the `hidden layer vector` as

$$Z = g(b^{(1)} + \mathbf{W}^{(1)} X) \in \mathbb{R}^m, \qquad \mathbf{W}^{(1)} = \begin{bmatrix} w_1^{(1)} \\ \vdots \\ w_m^{(1)} \end{bmatrix} \in \mathbb{R}^{m \times p}, \qquad b^{(1)} = \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_m^{(1)} \end{bmatrix} \in \mathbb{R}^m,$$

where the function g is applied to each component of the argument vector.

Single hidden layer neural network for regression

- In matrix notation, we can write the hidden layer vector as

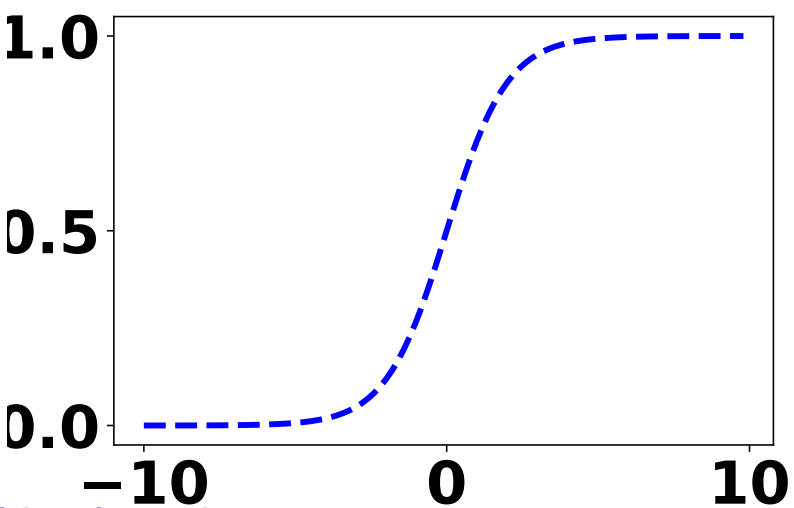
$$Z = g(b^{(1)} + \mathbf{W}^{(1)} X) \in \mathbb{R}^m, \qquad \mathbf{W}^{(1)} = \begin{bmatrix} w_1^{(1)} \\ \vdots \\ w_m^{(1)} \end{bmatrix} \in \mathbb{R}^{m \times p}, \qquad b^{(1)} = \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_m^{(1)} \end{bmatrix} \in \mathbb{R}^m,$$

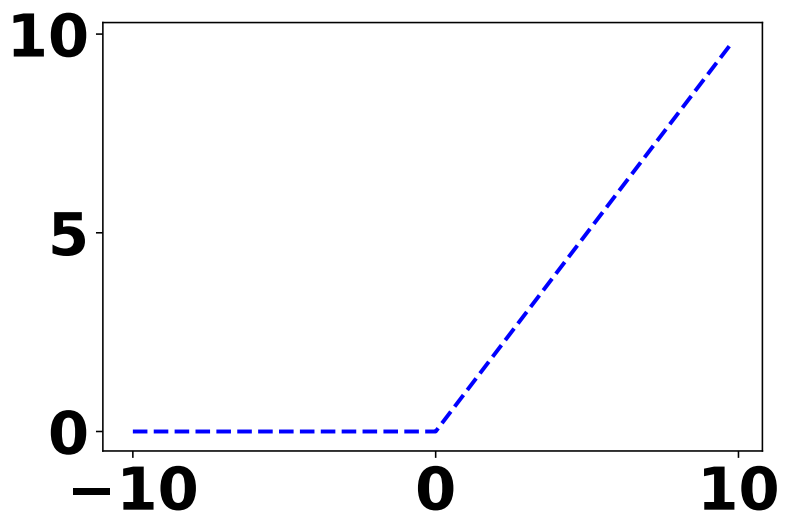
  where the function g is applied to each component of the argument vector.
- Traditionally, the activation function g was chosen to be the sigmoid function

$$\sigma(x) = \exp(x)/(1 + \exp(x)).$$

- In modern neural networks, the default recommendation for g is the rectified linear unit or ReLU given by

$$\text{ReLU}(x) = \max\{0, x\}.$$





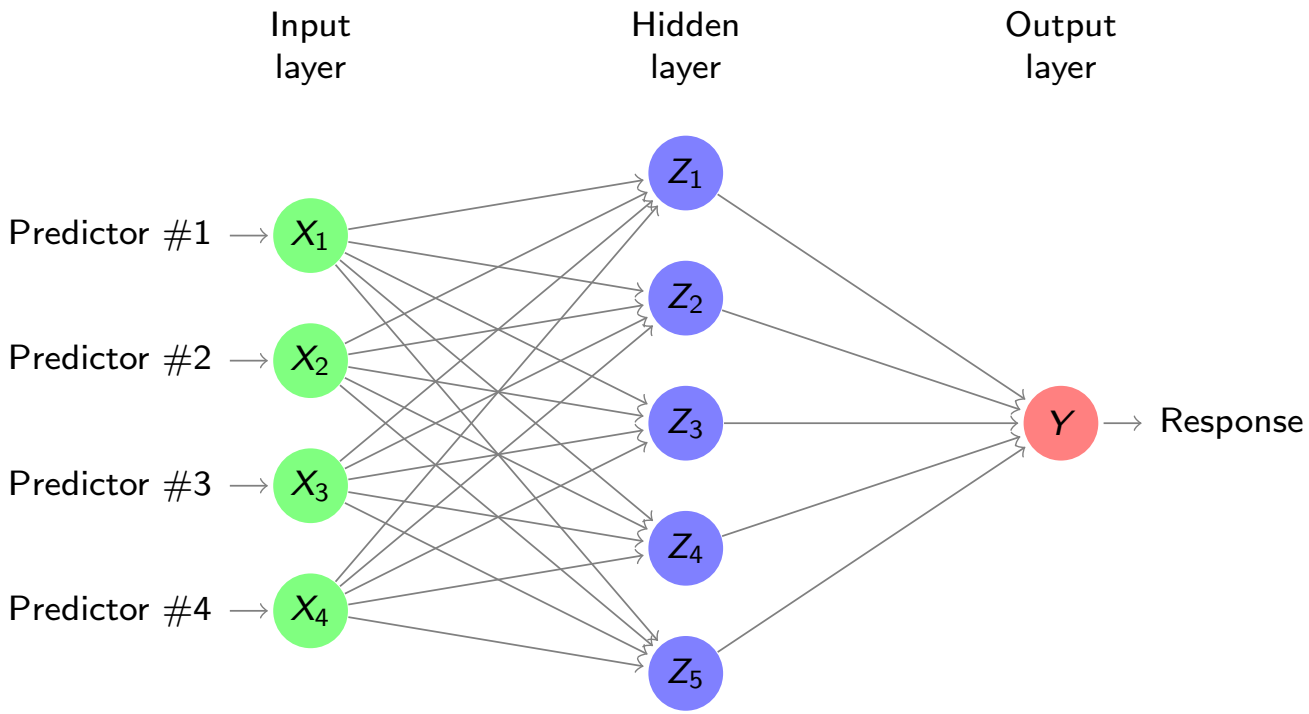
Single hidden layer neural network for regression

- In regression, the function h is typically the `identity` $h(z) = z$, that is,

$$\widehat{f}(X) = b^{(2)} + {w^{(2)}}^{\top} Z.$$

- Together with the `ReLU activation` $Z = \max\{0, b^{(1)} + \mathbf{W}^{(1)} X\}$, the `single hidden layer neural network` models the output Y in a non-linear way by

$$\widehat{f}(X) = b^{(2)} + {w^{(2)}}^{\top} \max\{0, b^{(1)} + \mathbf{W}^{(1)} X\}$$

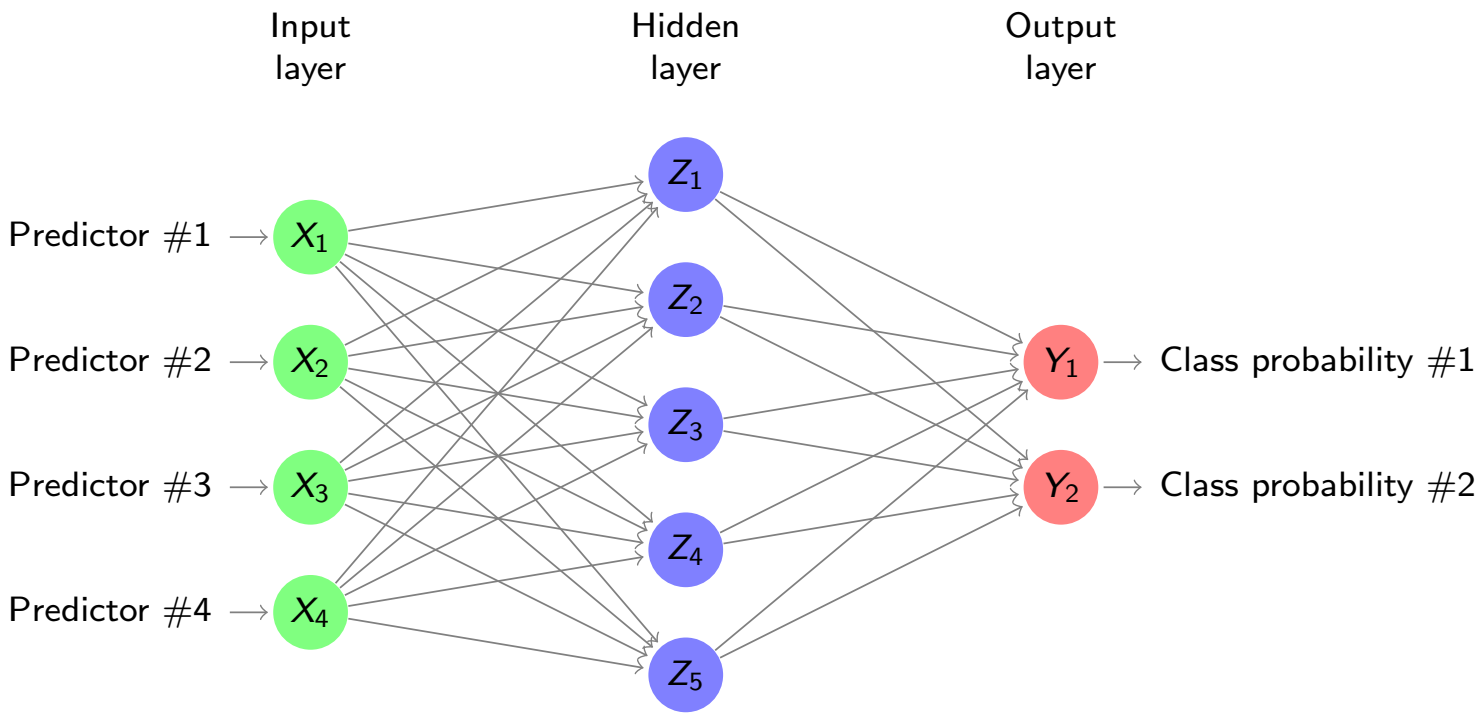


Single hidden layer neural network for classification

- If the output is qualitative with q classes $Y \in \mathcal{G} = \{1, \ldots, q\}$ then there are q output nodes, each standing for the probability $Y_i = f_i(X) = \Pr(Y = i \mid X)$, $i = 1, \ldots, q$.
- The neural network is then

$$Z = g(b^{(1)} + \mathbf{W}^{(1)} X)$$
$$\widehat{f}_i(X) = h_i(b^{(2)} + \mathbf{W}^{(2)} Z), \quad i = 1, \ldots, q.$$

with weight matrix $\mathbf{W}^{(2)} = (w_{ij}^{(2)}) \in \mathbb{R}^{q \times m}$ and bias vector $b^{(2)} = (b_1^{(2)}, \ldots, b_q^{(2)}) \in \mathbb{R}^q$, and some functions h_i.



Single hidden layer neural network for classification

- As for regression, the default choice for the `activation` is the `ReLU` $g(x) = \max\{0, x\}$.

Single hidden layer neural network for classification

- As for regression, the default choice for the `activation` is the `ReLU` $g(x) = \max\{0, x\}$.
- For classification, the functions h_i are the `softmax` functions

$$h_i(x_1, \ldots, x_q) = \text{softmax}(x_1, \ldots, x_q)_i = \frac{\exp(x_i)}{\sum_{j=1}^{q} \exp(x_j)}, \quad i = 1, \ldots, q.$$

- Note that this is the same function as used in `multinomial logistic regression`.

Single hidden layer neural network for classification

- As for regression, the default choice for the activation is the ReLU $g(x) = \max\{0, x\}$.
- For classification, the functions h_i are the softmax functions

$$h_i(x_1, \ldots, x_q) = \text{softmax}(x_1, \ldots, x_q)_i = \frac{\exp(x_i)}{\sum_{j=1}^{q} \exp(x_j)}, \quad i = 1, \ldots, q.$$

- Note that this is the same function as used in multinomial logistic regression.
- With these choices the single hidden layer neural network for classification models the posterior probabilities as

$$\widehat{f}_i(X) = \Pr(Y = i \mid X) = \text{softmax}(b^{(2)} + \mathbf{W}^{(2)} \max\{0, b^{(1)} + \mathbf{W}^{(1)} X\})_i.$$

- For a new input $x_0 \in \mathbb{R}^p$ we then use the Bayes classifier to predict the class

$$\widehat{G}(x_0) = \widehat{y}_0 = \arg \max_{i=1,\ldots,q} \widehat{f}_i(x_0).$$